ErpOne

Project Overview & Architecture Reference

Brand: ERP_One · SQL Database: MyApp · Solution: `ErpOne.slnx` · Generated 2026-07-14

1. What is ErpOne?

ErpOne is a web-based ERP for retail & distribution businesses (Indonesian SME/distributor context). It covers master data, purchasing, sales, warehouse/inventory, point-of-sale, finance (AR/AP/expense), reporting, and an operational dashboard. The UI blends English labels with Indonesian business terminology.

Tech stack

.NET 10 · Blazor Server (InteractiveServer) · EF Core · SQL Server · FluentValidation · ClosedXML (Excel) · QuestPDF (PDF) · Bootstrap + Bootstrap Icons · xUnit (+ SQLite in-memory) · ASP.NET Identity

2. Architecture (Clean Architecture)

Four projects with strict inward dependencies. Outer layers depend on inner; the Domain depends on nothing.

Layer	Project	Responsibility	Depends on
Domain	`ErpOne.Domain`	Entities, enums, domain rules. No framework deps.	—
Application	`ErpOne.Application`	Service interfaces, DTOs, FluentValidation validators, ports.	Domain
Infrastructure	`ErpOne.Infrastructure`	EF Core `AppDbContext`, service implementations, migrations, DI wiring.	Application, Domain
Web	`ErpOne.Web`	Blazor pages/components, authorization, menu, seeding, appearance.	Application, Infrastructure

Tests: `ErpOne.UnitTests` (domain/logic) and `ErpOne.IntegrationTests` (services over SQLite; references `ErpOne.Web` via `WebApplicationFactory<Program>`).

3. Folder organization (by menu header)

Within each project, files are grouped into subfolders that mirror the sidebar menu headers. **Folders are organizational only — namespaces are NOT nested** (e.g. a service under `Services/Master/` still lives in namespace `ErpOne.Infrastructure.Services`). This keeps zero-friction builds while giving a clear map.

```
Master/        Product, ProductCategory, Brand, Unit, Attribute, Warehouse,
               Tax, PaymentMethod, Supplier, Customer, Currency
Inventory/     Stock (StockMovement, ProductStock)
Transactions/  PurchaseOrder, GoodsReceipt, SalesOrder, DeliveryOrder
Cashier/       CashierShift, PosSale
Finance/       CashBank, Supplier/CustomerInvoice, Supplier/CustomerPayment&Receipt, Expense
Reports/       StockLedger, InventoryValuation, Sales, Purchase, GrossProfit
Dashboard/     Dashboard
Settings/      CompanySetting, ApprovalChain, NumberSequence, DocumentNumber, Log
```

Applies to `Domain/Entities/*`, `Application/*` (feature folders nested under headers), and `Infrastructure/Services/*`. `Application/Common`, `Reports`, `Dashboard` stay at their project root.

4. Data & persistence

- **DbContext:** `ErpOne.Infrastructure.Persistence.AppDbContext`. Entities configured inline in `OnModelCreating` (per-entity blocks).
- **Table naming:** physical tables use domain prefixes — `M_*` (master, e.g. `M_ProductVariants`), `T_*` (transactions, e.g. `T_DeliveryOrders`).
- **Migrations:** `dotnet ef migrations add <Name> --project src/ErpOne.Infrastructure --startup-project src/ErpOne.Web`, then `dotnet ef database update ...`.
- **Connection:** `ConnectionStrings:Default` in `appsettings.json` → SQL Server database `MyApp`.
- **Costing:** stock uses moving-average; COGS on delivery is snapshotted from `ProductVariant.CostPrice` at *Post* time (draft lines show 0 until posted).

5. Menu & navigation system

The left sidebar (light hover-expand icon rail) is **data-driven** from a single registry.

- **`AppMenus.cs`** (Web/Authorization) — declares `ResourceGroup`s (menu headers) each containing `AppResource`s. Each resource = `(key, label, icon, actions[])`. Actions are `AppAction` like `index/create/edit/delete/approve/post/void/export`.
- **`NavMenuBuilder.cs`** — builds the sidebar by convention: `Href = key.Replace('.', '/')`; visibility gate = `{key}.index`; icon = declared icon minus `-fill`. Irregular cases handled by an `Overrides` table (custom Href / action / always-visible / extra items).
- **Permissions** are auto-derived: `AppMenus.AllPermissions` = every `{resource}.{action}`. Helper: `AppMenus.Perm("master.brands","create")`.

6. Authorization & permissions

- Policy name = permission string = `{resource}.{action}` (e.g. `finance.ar-invoices.index`).
- Global `FallbackPolicy` requires an authenticated user (every page needs login unless `[AllowAnonymous]`).
- Pages gate with `@attribute [Authorize(Policy = "…")]`; buttons/cards gate with `<AuthorizeView Policy="…">`; forms re-check create/edit permission in `OnInitializedAsync`.
- **`BootstrapSeeder.cs`** (Web/Infrastructure) grants *all* `AppMenus.AllPermissions` to the admin role on startup (idempotent) and ensures the bootstrap admin user. *After adding a permission, restart + sign out/in so the admin's*

claims refresh.

7. Cross-cutting services

Concern	Service	Notes
Document numbering	`IDocumentNumberService` + `NumberSequence`	Prefixed sequences (e.g. APV/APP/ARV/ARR/EXP). Integration tests assert the sequence count.
Reporting/export	`ReportDocument` + `IReportExporter`	Excel via ClosedXML, PDF via QuestPDF. Read-only query services in `Application/Reports`.
Validation	FluentValidation	Validators registered via `AddValidatorsFromAssemblyContaining<CreateProductValidator>()`. Services throw `ValidationException`.
Approvals	`ApprovalChain` / `ApprovalService`	PO/SO submit → PendingApproval; empty chain auto-confirms. Void flows verify credentials + role.

8. UI design system

Token-based and theme-aware (light / `html[data-bs-theme="dark"]`). Accent & font are user-configurable via an Appearance popover (`--app-accent`, `--app-font` on `<html>`).

Container class	Used for	Tokens defined?
`.pi`	Index / list pages (KPIs, toolbar, chips, table, Pager)	Yes (scoped in app.css) + dark overrides
`.cf`	Master create/edit forms (card + grid)	Yes + dark overrides
`.pf .pf-form`	Transaction forms (line-items table)	Yes + dark overrides
`.pf .pf-detail`	Transaction detail pages (info grid, items, timeline)	Yes + dark overrides
`.dash` / `.landing`	Dashboard & Home (self-contained tokens)	Yes (defined in component CSS)

Token rule: design tokens (`--ink, --muted, --accent, --panel, --field, --line, --danger`) are defined *scoped to each page-container class* and bridge to `--app-accent`. A new top-level page container MUST define its own tokens + a `html[data-bs-theme="dark"] .yourcontainer` override, or colors resolve to nothing.

9. Functional modules

Area	Highlights
Master	Products (+variants, attributes, images), categories, units, brands, warehouses, taxes, payment methods, suppliers, customers, currencies.
Inventory	Stock levels (per variant × warehouse, moving-average cost), stock adjustments/opname. Filters: All / Low stock (≤5) / Out of stock.
Transactions	Purchase Order → Goods Receipt (stock in); Sales Order → Delivery Order (stock out + COGS). Statuses incl. PendingApproval.
Cashier	Cashier shift open/close; POS sale (revenue = line total, COGS = unit cost × qty).
Finance	Cash & Bank ledger; Supplier Invoice/Payment (AP); Customer Invoice/Receipt (AR, credit limit); Expense & categories; void-with-authorization.
Reports	Stock Ledger, Inventory Valuation (as-of-date), Sales, Purchase, Gross Profit — all with Excel/PDF export.
Dashboard	Hero revenue (today, accent gradient) + month-to-date, receivables/payables due, PO/SO pending, aging, inventory. Real 7-day trend via one query.
Settings	Users, roles, company profile, approval chains, document numbering, error log.

10. Build, run & test

```
# Build / test the whole solution
dotnet build ErpOne.slnx
dotnet test  ErpOne.slnx

# Run: from Visual Studio (IIS Express) or
dotnet run --project src/ErpOne.Web
```

Gotcha: while the app runs in Visual Studio/IIS Express, the `ErpOne.Web` DLLs are locked, so `dotnet build/test` of the solution fails with copy errors (MSB3021) — *not* a compile error. Integration tests reference `ErpOne.Web`, so **stop the running app before `dotnet test`**. Building `Domain/Application/Infrastructure` individually works while the app runs.

11. Conventions & gotchas

- **Git:** commit / merge / push are done *manually* by the developer. Tooling leaves changes in the working tree (uses `git mv` for renames).
- **Query translation:** integration tests run on SQLite — avoid SQL-Server-only translations (e.g. `EF.Functions.DateDiffDay`). Prefer date-threshold comparisons or in-memory aggregation for small sets.
- **Rupiah:** amounts shown with `ToString("N0")` / `N2`. (Indonesian locale formatting is a candidate future improvement.)
- **Numbering test:** when adding a `NumberSequence`, bump the count assert in `NumberSequenceServiceTests`.
- **Solution file** is `ErpOne.slnx` (not `.sln`).